

状態管理の関数呼び出し

これから、エネミーにいくつかの行動を実装していきますので、
ステートデザインパターンに沿って、行動制御を行いましょう。

THINK	...	これからの行動を考える状態
IDLE	...	待機状態
WANDER	...	ウロウロと移動する状態

```
EnemyRat.h
```

```
public:
```

```
// 状態
```

```
enum class STATE
```

```
{
```

```
    NONE,
```

```
    THINK,
```

```
    IDLE,
```

```
    WANDER,
```

```
    END
```

```
};
```

```
private:
```

```
    ~ 省略 ~
```

```
// 状態
```

```
STATE state_;
```

```
// 更新ステップ
```

```
float step_;
```

```
// 状態遷移
```

```
void ChangeState(STATE state);
```

```
void ChangeStateNone(void);
```

```
void ChangeStateThink(void);
```

```
void ChangeStateIdle(void);
```

```
void ChangeStateWander(void);
```

```
void ChangeStateEnd(void);
```

```
// 更新系
void UpdateNone(void);
void UpdateThink(void);
void UpdateIdle(void);
void UpdateWander(void);
void UpdateEnd(void);
```

【要件①】

cpp側にステートデザインパターンを実装すること。

- ・ 初期STATEはTHINKにすること

【目標①】

迷わず、スラスラとステートデザインパターンが実装できること。

よりコードを短くするために

switch文を使用して、
状態遷移関数、状態別更新関数を呼び出しているかと思いますが、
行動の種類が多くなればなるほど、コードが長くなってしまいます。

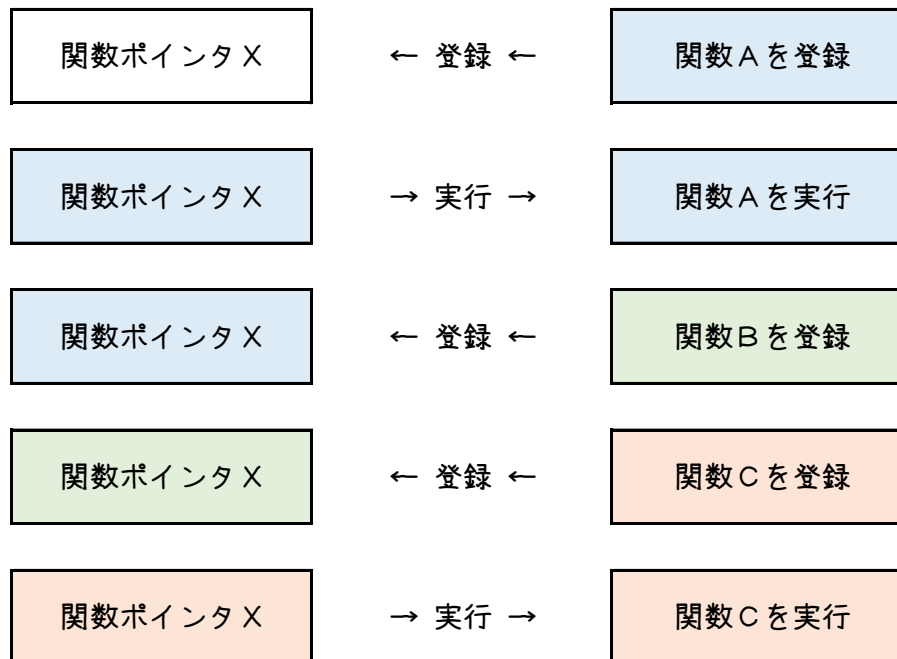
コードが長くなると、読みづらくなり、可読性が下がります。
特にUpdate関数は、Init関数と比較して、コードが煩雑になりやすく、
コードを読む負荷は、極力下げたいところです。

※卒業生の中には、caseが100以上あり、とても見づらい！
switch文は悪だ！という方もいらっしゃいます

これを解決するために、
関数ポインタや、function機能を使用します。

変数だけではなく、関数にもメモリアドレスが存在し、
このアドレスのことを関数ポインタといいます。

関数ポインタを特定の変数に格納しておくことで、
任意のタイミングで関数を実行することができます。



この仕組みを使用すれば、switch文を無くすことができますが、
関数ポインタよりも、より汎用的な機能を持つstd::functionでの
使用例を解説していきたいと思います。

std::function

呼び出し可能オブジェクトを格納できる多目的コンテナ。
関数ポインタの方が軽量だが、便利なたくさんの機能がある。

```
EnemyRat.h
```

```
#include <functional>
```

```
private:
```

```
// 状態
```

```
STATE state_;
```

```
// 状態管理(状態遷移時初期処理)
```

```
std::map<int, std::function<void(void)>> stateChanges_;
```

```
// 状態管理(更新ステップ)
std::function<void(void)> stateUpdate_;
```

EnemyRat.cpp

```
void EnemyRat::InitPost(void)
{

    // 状態遷移初期処理登録
    stateChanges_.emplace(static_cast<int>(STATE::NONE),
        std::bind(&EnemyRat::ChangeStateNone, this));
    stateChanges_.emplace(static_cast<int>(STATE::THINK),
        std::bind(&EnemyRat::ChangeStateThink, this));
    stateChanges_.emplace(static_cast<int>(STATE::IDLE),
        std::bind(&EnemyRat::ChangeStateIdle, this));
    stateChanges_.emplace(static_cast<int>(STATE::WANDER),
        std::bind(&EnemyRat::ChangeStateWander, this));
    stateChanges_.emplace(static_cast<int>(STATE::END),
        std::bind(&EnemyRat::ChangeStateEnd, this));

    // 初期状態設定
    ChangeState(STATE::THINK);

}

void EnemyRat::UpdateProcess(void)
{

    // 状態別更新
    stateUpdate_();

}

void EnemyRat::ChangeState(STATE state)
{

    state_ = state;

    // 各状態遷移の初期処理
    stateChanges_[static_cast<int>(state_)]();

}
```

```

}

void EnemyRat::ChangeStateNone(void)
{
    stateUpdate_ = std::bind(&EnemyRat::UpdateNone, this);
}

void EnemyRat::ChangeStateThink(void)
{
    stateUpdate_ = std::bind(&EnemyRat::UpdateThink, this);
}

void EnemyRat::ChangeStateIdle(void)
{
    stateUpdate_ = std::bind(&EnemyRat::UpdateIdle, this);
}

void EnemyRat::ChangeStateWander(void)
{
    stateUpdate_ = std::bind(&EnemyRat::UpdateWander, this);
}

void EnemyRat::ChangeStateEnd(void)
{
    stateUpdate_ = std::bind(&EnemyRat::UpdateEnd, this);
}

```

解説

- ・ 引数無し、返り値無しの関数を登録できるコンテナ

```

// 状態管理(更新ステップ)
std::function<void(void)> stateUpdate_;

```

引数int、返り値floatの場合

```

std::function<float(int)> func;

```

- functionに関数を登録する方法

```
stateUpdate_ = std::bind(&EnemyRat::UpdateIdle, this);
```

bind関数を使用して、クラス名::関数名を第一引数に記述する。
thisは、自分自身のクラスのポインタ。

- functionに登録した関数を実行する方法

```
stateUpdate_();
```

- Change系とUpdate系で書き方を変えているのはなぜ？

```
std::map<int, std::function<void(void)>> stateChanges_;
```

Initの段階で、STATEごとにmap連想配列に対応関数を登録しておくことで、関数名を意識しなくて良いというメリットがあります。

一方、連想配列の中を毎回アクセスすることになりますので、
毎フレーム実行行おうような、Updateには使用しない方が良いでしょう
という理由で、Change系とUpdate系で書き方を変えています。

しかし、微々たる差にはなりますので、
2種類あるうちのどちらかに統一しても大丈夫です。

基底クラスへの汎化

Enemyだけではないのですが、派生クラス側で状態遷移を実装する時、`enum STATE` を `EnemyBase` (基底クラス) 側に記述したくなりますが、実装する `Enemy` にもよりますが、それぞれの `Enemy` で状態 (`STATE`) が一致しないケースが多いため、派生クラス側で実装しておいた方が良いです。

しかし、`EnemyBase` (抽象) から状態 (`STATE`) を取得しようとする、派生クラスにキャストしないといけなかったり、先ほどの関数ポインタや `function` の仕組みを派生クラスで毎回記述するのも、省略化したい場合は以下のように汎化することができます。

`enum` は、結局 `int` なので、`EnemyBase` 側では `int` で管理する。

前述の `map` 連想配列のキー (`first`) が `STATE` ではなく、`int`なのはそのため。

```
std::map<int, std::function<void(void)>> stateChanges_;
```

```
EnemyBase.h
```

```
#include <functional>
```

```
protected:
```

```
// 状態管理
```

```
int stateBase_;
```

```
// 状態管理(状態遷移時初期処理)
```

```
std::map<int, std::function<void(void)>> stateChanges_;
```

```
// 状態管理(更新ステップ)
```

```
std::function<void(void)> stateUpdate_;
```

```
～ 省略 ～
```

```
// 状態遷移
```

```
void ChangeState(int state);
```

EnemyBase.cpp

```
void EnemyBase::ChangeState(int state)
{
    stateBase_ = state;

    // 各状態遷移の初期処理
    stateChanges_[stateBase_]();
}
```

EnemyRat.h

```
#include <functional>
```

と

```
// 状態管理(状態遷移時初期処理)
std::map<int, std::function<void(void)>> stateChanges_;

// 状態管理(更新ステップ)
std::function<void(void)> stateUpdate_;
```

を削除。

EnemyRat.cpp

```
void EnemyRat::ChangeState(STATE state)
{
    state_ = state;
    EnemyBase::ChangeState(static_cast<int>(state_));
}
```

こういった形で基底クラスに持たせれば、ある程度汎化することができます。
そして、共通するSTATEがあれば、EnemyBaseに記述し、派生クラスから
呼び出すことでスッキリすると思います。